

CANDY DIALOGUE ENGINE

Word Substitution

1.0.0

The Word Substitution feature allows you to swap or insert words with another variant inside dialogue lines, based on actor properties. The most common usage is to replace gendered words correctly. For example, consider the following sentence:

"He's very smart! I wish I was like him!"

While the sentence itself is grammatically correct, what if it sometimes refers to a male character, and sometimes to a female character? Obviously, in the latter case, the sentence should change:

"She's very smart! I wish I was like her!"

This guide will explain how to do this.

Basics

Usage

Candy Dialogue Engine provides two variables in **Candy_Properties.gd** -- **substitution_symbol** and **substitution_separator**. By default, these are set to the symbols '•' and '|' respectively (you may change them to characters that are more convenient to type and which you'll never use in any other context).

Meanwhile, **Candy_Database.gd** features the **substitution_table**, which contains a list of keywords that can be substituted:

```
#^ Gender substitution dictionary:
var substitution_table := {
  "Default*": {
    "he, she": [
      "auto",
      {"Gender": {
        "M": "he",
        "F": "she",
      }},
    ],
    "him, her": [
      "auto",
      {"Gender": {
        "M": "him",
        "F": "her",
      }},
    ],
  },
}
```

A bit of vocabulary here:

"**Default***" is a **variant table**. More on that later.

"**he, she**" and "**him, her**" are **keywords**. Combined with their respective arrays, they are called **substitution rules**.

"**auto**" is a **case rule**. More on that later.

"**Gender**" is a **property**.

"**M**" and "**F**" are **values**.

"he", "she", "him" and "her" are **substitute words**.

With that in mind, here is how to use Word Substitution:

All you have to do is to combine both the **symbols** and the **keywords** in spoken text, like this:

"•He|Player•'s very smart! I wish I was like •him|Player•!"

The '•' symbols tell the engine where a word substitution starts and ends. The '|' symbol tells the engine where to separate the **keyword** ('He' and 'him') from the referenced actor ('Player').

When the engine encounters such formatting, it looks up the **keywords** inside the **substitution_table**, and it looks for the Player's "**Gender**" key in the **actors** dictionary. It then uses the value (e.g. "F") to figure out what word to replace 'he' and 'him' by: 'she' and 'her'.

When the dialogue plays, the text will appear like this:

"She's very smart! I wish I was like her!"

As usual with data in Candy DE, any part of the substitution syntax can be replaced by variables:

•£globals.variable_1|£globals.variable_2•

Properties Dictionaries & Arrays

If the property you're trying to access is inside another dictionary or array, you can still use it for comparison, like this:

"Stats.Gender" instead of "Gender"

When you write **•he|Alice•**, it will check the value of `candy.actors_dictionary["Alice"]["Stats"]` `["Gender"]`.

What Word Substitution cannot do at this time is making comparisons to values outside the **actors** dictionary.

Value Types

Property values can be a string, an integer, a boolean or a float. However, by default, Candy DE will read them all as strings. That means, if a value is '10' (integer) or 'true' (boolean), Candy DE will read them as "10" and "true" (strings).

This is not a problem in most cases, but it means that the Word Substitution feature sees a value of '10' (integer) and "10" (string) as the same. Substitutions will not work if you expect different results based on variable type.

There's a solution to this in case variable types should matter, but it will be explained in the advanced section of this guide.

Features

Automatic Case

This might have escaped you in the example above, but Candy DE automatically formats the case of the substituted words to match how you originally wrote them in the spoken text.

Lowercase → Writing "he" displays "he", "she", etc.

Uppercase → Writing "HE" displays "HE", "SHE", etc.

Capitalized → Writing "He" displays "He", "She", etc.

This way, you don't need to add repeated entries in the `substitution_table` for each case format.

However, if you want to always force a specific case (or just disable automatic case), there is also an option:

```
#^ Gender substitution dictionary:
var substitution_table := {
  "Default*": {
    "he, she": [
      "auto",
      {"Gender": {
        "M": "he",
        "F": "she",
      }},
    ],
  },
}
```

Inside the **keyword** array, in the first position (index 0) add a string:

"auto", "a", or empty string ("") → automatic formatting, as described above.

"lowercase", "lower", "low", or "l" → always change to lowercase.

"uppercase", "upper", "up", or "u" → always change to uppercase.

"capitalized", "cap", or "c" → always change to capitalized form.

"custom", or "x" → always write the replacement words exactly as they are in the table.

As a reminder, this is called a **case rule**. The case you write these options in doesn't matter, so feel free to write "Capitalized", "C" or even "cAPitAlIZEd, if you find it more intuitive.

Variants

If you don't know what Variants are in Candy DE, please look at the [Variants & Translations](#) guide now.

The Word Substitution feature adapts to variants: in the `substitution_table`, the various replacement rules are always located under a variant, such as **"Default"**:

```
#^ Gender substitution dictionary:
var substitution_table := {
  "Default*": {
    "he, she": {
      "auto",
      {"Gender": {
        "M": "he",
        "F": "she",
      }},
    },
  },
},
```

If you still haven't read the **Variants & Translations guide**, all spoken text for a spoken line is deemed a variant, and "Default" is the name of the original variant. Spoken lines can have multiple variants, and other variants are custom-named by yourself (e.g. "Default_#2", "French", etc.).

This is important if you provide multi-lingual support in your game: each language can have its own substitution rules.

Notice the asterisk (*) at the end of **"Default*"**, in the image above: it means that any variant that starts with "Default" can use those substitution rules. So not only could the "Default" variant use them, but also "Default_#2" and such.

If you want variants to use rules only if their name is an exact match, then just omit the asterisk.

Of course, you'll probably want some rules to apply to all variants and languages. In that case, simply place those rules under a variant key named **"*"** (asterisk).

Note that if an entry matches multiple variant filters that have the same entry (e.g. "French_1_A" matches **"*"**, **"French*"** and **"French_1*"**), the closest match (the most specific) will be used (i.e. **"French_1*"** in this case).

Exact matches will always be prioritized. For example, if 'Alice' is found under **"French_1"** and **"French_1*"**, the variant "French_1" will use the 'Alice' entry under **"French_1"**.

Multiple Keys

As you may have noticed, inside the substitution_table, keys can feature multiple keywords, separated by commas (,) such as "he, she" or "him, her".

This way, whether you write **"•he!Alice•"** or **"•she!Alice•"**, the lookup is the same. This can help make dialogue more intuitive and readable when editing, especially compared to less evocative options such as **"•pronoun!Alice•"**.

Just make sure that keywords are exclusive to a single **substitution_table** key, otherwise the engine might lookup the wrong table.

Roles

Actor roles are also supported, and you'll probably use them often since you normally know in advance the gender of any actor.

Simply prefix them with the **role_symbol** (default '°'), which is how they should be named anyway:

`"•he|°enemy_1•"`

Candy DE will find the associated actor in the **roles** dictionary, and lookup the right substitution values.

Actor Properties

While the previous examples used gender, you can actually refer to any of an actor's properties in the **actors** dictionary to guide word substitution. For example:

```
"cl": {  
    "auto",  
    {"Class, Gender": {  
        "Warrior": "Warrior",  
        "Sorcer, M": "Sorcerer",  
        "Sorcer, F": "Sorceress",  
    }},  
},
```

In the above example, writing `"•cl|Alice•"` will lookup Alice's "Class" key in the **actors** dictionary, and will use its value to figure out whether to replace 'cl' with "Warrior", "Wizard" or "Rogue".

Advanced Features

Just a forewarning, but what follows might make the Word Substitution feature seem complicated.

It really isn't: so far, this guide showed you how to substitute specific words based on a property of an actor. It's great for easily swapping gendered words, which is something many games struggle with. For most projects, this is all one needs to know.

You'll likely never need any of the more advanced options that will be explained below. They're entirely optional, provided to be helpful when you have more complex needs.

It doesn't hurt to take a glance, just to know what's available if you ever need it, but you don't **have** to learn or even take into consideration any of this until you actually want to use it.

Multiple Properties

You can assign multiple **properties** to a **keyword** in the **substitution_table**, like this:

```
"id": [
  "auto",
  {"Origins": {
    "France": "French",
    "England": "English",
    "Italy": "Italian",
  }},
  {"Language": {
    "FR": "French",
    "ENG": "English",
    "IT": "Italian",
  }},
],
```

When a substitution rule features multiple **properties**, simply use the following format to indicate the **property** you wish to compare:

•id|Alice|Language•

Alternately, you can use the array index of a **property** instead of writing its full name:

•id|Alice|2•

Keep in mind that the case rule (e.g. "auto") occupies index 0 in the array, so **properties** start at index 1. This has the benefit of being more intuitive, especially for non-coders.

Note that if you don't specify a **property** or an index, the engine will default to the first **property** in the array. So writing **•id|Alice•** will still work: in the example above, it will use the "Gender" **property**.

Joined Properties

You can also lookup any number of **properties** at once on an actor. For example:

```
{ "Class, Gender": {  
  "Sorcer, M": "Sorcerer",  
  "Sorcer, F": "Sorceress",  
}}
```

In the example above, if "Class, Gender" isn't the first value in the 'cl' array, you would write the following in text:

•cl|Alice|Class, Gender•

(Or replace 'Class, Gender' with its index for brevity)

Next, for the lookup to work, the property **values** must follow the same order as the **properties**.

That is, in the example above, "**Sorcerer, M**" will work, as it will compare "Sorcerer" to "Class" and "M" to "Gender". But "**M, Sorcerer**" will fail, as the order is incorrect: the engine will attempt to check if "Sorcerer" = "M", and "Gender" = "Sorcerer".

There's also an option to 'skip' properties when needed: just write an underscore (_):

```
{ "Gender, Class": {  
  "_ , Warrior": "Warrior",  
  "M, Sorcer": "Sorcerer",  
  "F, Sorcer": "Sorceress",  
}}
```

This tells the engine that there's no value for "Gender", therefore: "Warrior" is compared to "Class", but no comparison to "Gender" is made.

If you have a comparison with many joined properties, you can use as many empty values as needed:

"_ , M, _ , _ , English" → If there are 5 or more **properties**, the first property is compared to an empty value (the property is skipped), the second **property** is compared to "M", the third and fourth **properties** are compared to empty values, and the fifth **property** is compared to "English".

Operators

You might wish to use a few operators sometimes:

```
"id": {  
  "auto",  
  {"Age, Age": {  
    ">=|18": "adult",  
    "<=|18, >|11": "Teenager",  
    "<=|18, <=|11": "Child",  
  }},  
}
```

In this example, we want to substitute 'id' for either "Adult", "Teenager" or "Child" depending on the actor's age. Therefore, to avoid listing all possible ages, we can just use >, <, >= and <= operators (these are the only available operators at this time).

Operators must be separated from the values by the **separator_symbol** (default '|'). As an example, if you write ">18" instead of ">|18", Candy DE will check if the value of the actor's "Age" property matches a string of ">18" rather than checking if the age is greater than 18.

Notice also that you can check the same property multiple times. In the example above, this is necessary for "Teenager" and "Child" because the "AND" or "OR" operators are not available with the substitution feature (we said we were keeping it simple!). Therefore, we first check whether Age is smaller or greater than 18, and we then check if it's smaller or greater than 11.

Unused Joined Properties

Sometimes, you don't need to actually compare all the listed joined properties. For example:

```
"cl": {  
  "auto",  
  {"Class, Gender": {  
    "Warrior": "Warrior",  
    "Sorcer, M": "Sorcerer",  
    "Sorcer, F": "Sorceress",  
  }},  
}
```

Here, we need to compare both Class and Gender for "Sorcerer" and "Sorceress", since those class names change depending on the actor's gender. But "Warrior" is written the same whether the actor is male or female, so it doesn't need to be compared to Gender.

Thankfully, the 'Warrior' comparison ("Warrior: "Warrior") works fine:

"Warrior" is the first value, so it's compared to the first property: "Class". Since there's no second value, "Gender" is ignored.

Keep in mind that this only works because "Class" is the first property in the list. If the property string was "Gender, Class", the comparison would fail because "Warrior" would be compared to "Gender". Properties and values are always paired in left-to-right order for comparisons, and excess properties or values are ignored.

Value Types - Advanced

As mentioned previously, all values are internally converted to strings when Candy DE reads them. Most of the time, this is not a problem because a property will rarely alternate between various variable types.

In other words, you will likely never have an actor property that can be either an integer, a boolean or a string, where that distinction matters for the purposes of the Word Substitution feature.

If you do happen to encounter such a situation, there's an easy solution: use the **separator_symbol** to indicate the expected type:

```
"10|int, true|bool, 20"
```

- '10' will be read as an integer, and treated as a different value than '10' as a string.
- 'true' will be read as a boolean, different from 'true' as a string.
- '20' will be read as a string.

It works with operators too, no conflict at all:

```
">|10|int"
```

Type Comparisons

Finally, you may also want to substitute words based on the *variable type* of a value, rather than the value itself.

This can be done simply. Instead of an actual value, write this:

```
"int|type"
```

'type' will indicate to Candy DE that it is supposed to compare the type of the variable, not its value. It will then check the type you indicated (e.g. 'int') and see if it matches the given property it's making a comparison for.

This is probably a very unlikely occurrence, but we anticipate developers to get very creative with the tools they have and we'd rather provide options than impose unnecessary restrictions.